

Week 7 logistic regression and classification

1. What is the sigmoid function? What does it map the input to? Is it monotone? What do we use it to in the logistic function?

The sigmoid function is $1/(1+e^{-z})$ where z is $x*w$. It maps the input between 0 and 1. It is monotone. We use it in logistic regression to predict the probability of a value being in class 1.

It shows the ideal decision boundary

2. What is the derivative to the sigmoid function?

$y(1-y)$ where y is the sigmoid function.

3. What is logistic regression? Is logistic regression a classifier or regression?

Logistic regression is a statistical method used to analyze and model the relationship between a dependent binary variable (i.e., a variable with only two possible outcomes) and one or more independent variables. It is a type of regression analysis that uses the logistic function (also known as the sigmoid function) to model the probability of the dependent variable taking on a particular value (i.e., 0 or 1).

The logistic function maps any input to a value between 0 and 1, which can be interpreted as a probability. In logistic regression, the independent variables are used to predict the probability of the dependent variable taking on the value of 1. This is done by fitting a logistic curve to the data, which can be used to estimate the probability of the dependent variable based on the values of the independent variables.

Logistic regression is probability based. How likely is $P(t=1|x)$ and $P(t=0|x)$

4. What are the steps to predict in logistic regression? What is the typical threshold?

Find the sum of weighted input and apply the sigmoid function to the sum/ dot-product. The threshold is typically 0.5, so if the answer is above, it is class 1, else it is class 0.

$$\bullet \sigma(y) = \frac{1}{1+e^{-y}} > 0.5$$

5. How do we calculate the probability of x belonging to class 1 $P(t=1|x)$? how do we calculate $P(t=0|x)$

We use the sigmoid function to predict the probability $P(t=1|x) = 1/(1+e^{-w*x})$ and $P(t=0|x) = 1 - P(t=1|x)$

6. What is the goal of training?

The goal of training is to maximize $P(t|x)$ of all training instances. To do this we want to minimize the loss function.

7. What is cross entropy loss and how do we calculate it?

Cross entropy loss maximizes the joint probability of the predictions we make. $\text{Sum}(-\log(y^t(1-y)^{(1-t)}))$

$$\sum_{i=1}^N -\log P(t^{(i)}|\vec{x}^{(i)})$$

$$P(t|\vec{x}) = y^t(1-y)^{(1-t)}, \text{ whether } t = 1 \text{ or } t = 0$$

8. What is the chain rule for derivatives?

The chain rule is a fundamental rule of calculus that is used to find the derivative of a composite function. In calculus, a composite function is a function that is made up of two or more functions. The chain rule provides a way to find the derivative of such functions.

In simple terms, the chain rule states that the derivative of a composite function is equal to the derivative of the outer function evaluated at the inner function, multiplied by the derivative of the inner function.

Mathematically, if we have a composite function $f(g(x))$, the chain rule states that:

$$(d/dx) f(g(x)) = f'(g(x)) * g'(x)$$

where $f'(g(x))$ represents the derivative of the outer function f evaluated at the inner function $g(x)$, and $g'(x)$ represents the derivative of the inner function $g(x)$.

9. How do we find the derivative in logistic regression?

We find the derivative by finding the derivative of loss function. In the end the derivative is

$$w := \ln(X.T @ (y-t))$$

where $y = \text{logistic}(x@w)$

$$\bullet (\nabla f)_i = \frac{\partial}{\partial w_i} L_{CE}(X, T, w) = \sum_{j=1}^N -(t_j - y_j) x_{j,i}$$

$$- n * -(t-y)x$$

$$-n * (-t+y)x$$

$$-n(y-t)x$$

10. What is the difference in using cross-entropy loss or MSE when calculating loss in logistic regression?

Cross-entropy loss and Mean Squared Error (MSE) are two commonly used loss functions for logistic regression, and they have some important differences.

Cross-entropy loss is generally preferred over MSE for logistic regression because it has some desirable properties that make it well-suited for classification problems. Specifically, cross-entropy loss takes into account the probability distribution of the predicted values, while MSE only considers the distance between the predicted and actual values.

In logistic regression, we are trying to predict the probability that a given input belongs to a certain class. Cross-entropy loss measures the difference between the predicted probability distribution and the true probability distribution. It penalizes the model more when it makes large mistakes and less when it makes smaller ones. This makes it more effective for classification tasks where the goal is to accurately predict the class labels.

On the other hand, MSE is a measure of the average squared difference between the predicted and actual values. It is commonly used for regression problems where the goal is to predict a continuous value. MSE does not take into account the probability distribution of the predicted values and can result in suboptimal performance when used for classification tasks.

In summary, while both cross-entropy loss and MSE can be used for logistic regression, cross-entropy loss is generally preferred for classification tasks because it takes into account the probability distribution of the predicted values, while MSE does not.

11. What is batch training and stochastic gradient descent? Also what is mini-batch training?

Batch training, stochastic gradient descent, and mini-batch training are three common techniques used in machine learning to train models.

Batch training involves updating the model parameters based on the average of the gradients of the loss function calculated over the entire training dataset. In other words, the model weights are updated after processing all the training examples in a single batch. Batch training can be computationally expensive, especially for large datasets.

Stochastic gradient descent (SGD) is a variant of batch training that updates the model parameters after processing each training example or a small random subset of the training dataset. This makes SGD faster and more computationally efficient than batch training. However, the updates to the model parameters can be noisy and less stable than batch training.

Mini-batch training is a compromise between batch training and SGD. It involves updating the model parameters based on the average of the gradients of the loss function calculated over small batches of training examples. These batches are usually selected randomly from the training dataset. Mini-batch training provides a balance between the stability of batch training and the efficiency of SGD. It is the most commonly used training method in deep learning.

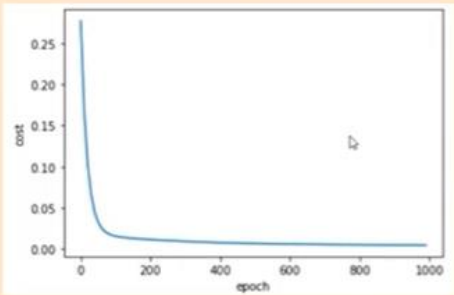
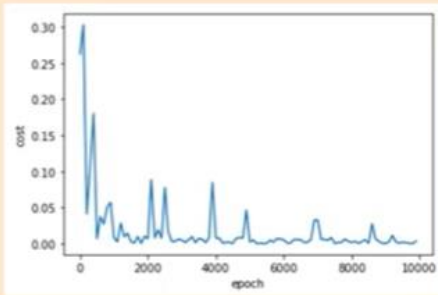
In summary, batch training updates the model parameters after processing the entire training dataset, stochastic gradient descent updates the model parameters after processing each training example or a small random subset of the training dataset, and mini-batch training updates the model parameters after processing small batches of training examples. Each of these techniques has its own advantages and disadvantages, and the choice of which technique to use depends on the specific problem and dataset being studied.

Batch training: Given a set of weights, one predicts the output values for all items in the training set. One then computes the loss and the gradient for this whole set and updates the weights accordingly. One then repeats this process a number of times.

Stochastic gradient descent: One calculates the loss and gradient for one item in the training set at a time, and repeat. Stochastic indicates that the next item to process is chosen randomly. This is in contrast to a sequential procedure where one goes through the data set a couple of times in a set order.

Mini-batch is an in-between the two others. One first decides on a batch size, for example 8 datapoints. The whole dataset is split into such mini-batches. In one epoch, one processes all the mini-batches. For each mini-batch, one calculates output, loss and gradient and update the weights accordingly. One then proceeds to the next mini-batch. One might consider shuffling the data points between the epochs

Batch Gradient Descent	Stochastic Gradient Descent (SGD)	Mini batch gradient descent
Use all training samples for one forward pass and then adjust weights	Use one (randomly picked) sample for a forward pass and then adjust weights	Use a batch of (randomly picked) samples for a forward pass and then adjust weights

Batch Gradient Descent	Stochastic Gradient Descent (SGD)
Use all training samples for one forward pass and then adjust weights	Use one (randomly picked) sample for a forward pass and then adjust weights
Good for small training set	Good when training set is very big and we don't want too much computation
	

12. What is one-hot encoding?

One-hot encoding transform categorical data to numeric data. Each categorical variable is converted into a binary vector with a length equal to the number of unique categories. Each unique category is assigned a unique integer index and the corresponding element in the vector is set to 1, while all the other elements are set to 0. For example, red, green, and blue are one-hot encoded. Red has the vector [1,0,0], green has [0,1,0] and blue has [0,0,1]

13. What is one-vs-rest

One-vs-rest is a multi-class classifier. It classifies one datapoint to a class c out of several classes

It first makes one classifier (for example logistic regression) for each class. Each classifier predicts a score. It assigns the class with highest score/probability.

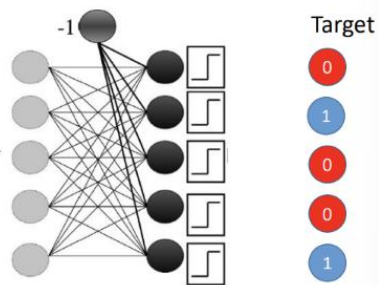
14. What is multi-label perceptron?

It is a multi-label classifier. In multi-label classification, each input sample may belong to one or more classes, and the task is to predict the presence or absence of each class label for the input.

The multi-label perceptron is a variation of the single-layer perceptron that is adapted for multi-label classification tasks. It consists of an input layer, a single output layer, and multiple binary classifiers for each label. Each binary classifier uses a sigmoid activation function to output a probability score indicating the likelihood of the corresponding label being present in the input sample.

Multi-label perceptron

- Marsland's description of perceptron:
 - Possible targets: (0,1,0,0,1)
 - $y_j = g(z_j) = \begin{cases} 1 & \text{if } z_j > 0 \\ 0 & \text{if } z_j \leq 0 \end{cases}$
 - Loss is 0-1 loss for each j
- Describes a multi-label classifier
 - Each y_j depends only on the $w_{h,k}$ where $k = j$
 - This could have been described as n independent binary perceptrons



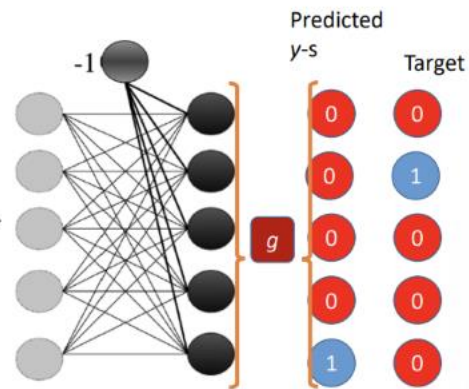
68

15. What is multi-class perceptron?

The multi-class perceptron is a variation of the single-layer perceptron that is adapted for multi-class classification tasks. It consists of an input layer, a single output layer, and multiple binary classifiers for each class label. Each binary classifier uses a softmax activation function to output a probability score indicating the likelihood of the input sample belonging to the corresponding class.

Multi-class perceptron

- The target contains one 1, the rest are 0s
- $g(z_1, z_2, \dots, z_n) =$
- $\text{argmax}(z_1, z_2, \dots, z_n)$
 - The index with the max value
- The update rule (0-1 loss)
 - $w_{i,j} = w_{i,j} - \eta(y_j - t_j)x_i$
 - will correct for $j = 2$ and $j = 5$
 - leaves the other weights unaltered



16. What is multi-output linear regression?

N input and n output nodes. Several linear regression with one output each.

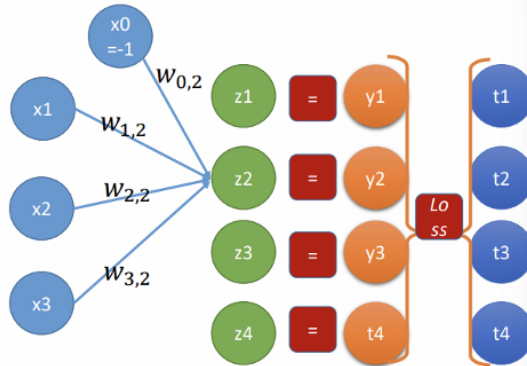
Multi-output linear regression is a type of regression analysis that models the relationship between multiple input variables and multiple output variables. It is an extension of the simple linear regression model that is used when there are multiple output variables.

In multi-output linear regression, the model assumes that the output variables are a linear combination of the input variables, where the coefficients of the linear combination are the model parameters. The model can be represented by a matrix equation of the form $Y = XW + E$, where Y is a matrix of the output variables, X is a matrix of the input variables, W is a matrix of the coefficients, and E is a matrix of the error terms.

Multi-output linear regression

- $y_j = g(z_j) = z_j$
- MSE-loss:

$$\sum_{k=1}^N (\sum_{j=1}^n (y_{k,j} - t_{k,j})^2)$$
 - n output nodes
 - N input items
- y_j independent of $w_{h,k}$ $h \neq k$,
 - hence corresponds to n independent models
 - (Gets more interesting for multi-layer networks)



70

17. What is Multinomial logistic regression

Multinomial logistic regression is a type of regression analysis used for modeling the relationship between a set of predictor variables and a categorical outcome variable with more than two categories. It is an extension of the binary logistic regression model, which is used for modeling a binary outcome variable.

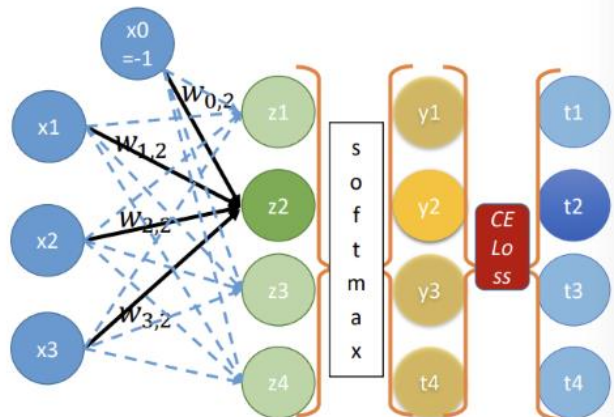
In multinomial logistic regression, the model estimates the probability of each category of the outcome variable given the values of the predictor variables. The model is based on a set of linear combinations of the predictor variables, where each category of the outcome variable is associated with a set of coefficients.

Predikerer hvilken klasse den mest sannsynlig tilhører

Multinomial Logistic Regression

- $z_j = \sum_{i=0}^n w_{i,j} x_i$
- Apply the softmax-function, S , where

$$y_j = (S(z_1, \dots, z_n))_j = \frac{e^{z_j}}{\sum_{k=1}^n e^{z_k}}$$
- Observe:
 - y_j depends on all the z_k
 - If $z_h > z_k$ then $y_h > y_k$
 - $0 < y_j < 1$
 - $\sum_{j=1}^n y_j = 1$
 - A probability distribution
 - $P(C_j | \vec{x}) = \frac{e^{\vec{w}_j \cdot \vec{x}}}{\sum_{k=1}^n e^{\vec{w}_k \cdot \vec{x}}}$



71